



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

DD/MM/2026

Análisis y diseño de sistema 2

Escuela de Ingeniería de Ciencias Y Sistemas

NOMBRE DEL TUTOR

UNIDAD #:3

Patrones de arquitectura

Anuncios Importantes



Anuncio 1 (Ingresa los anuncios importantes como lo sean, cortos, entregas, practicas, proyectos, horarios de calificación, etc.)



Anuncio 2



Anuncio 3



Anuncio 4



Anuncio 5

Agenda



Dudas del proyecto



Teoría



Caso práctico

COMPETENCIA(S) QUE DESARROLLAREMOS



Recuerda la importancia de las decisiones de diseño y arquitectura mediante el análisis de casos prácticos y ejemplos de sistemas reales para fortalecer la capacidad de aplicar soluciones coherentes y efectivas en proyectos de software



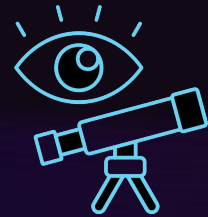
VALOR DE LA SEMANA

El pensamiento crítico es esencial al seleccionar y aplicar patrones de arquitectura, ya que permite evaluar de forma objetiva los requerimientos del sistema, las ventajas y desventajas de cada enfoque arquitectónico, y tomar decisiones informadas que impactan en la escalabilidad, flexibilidad y mantenibilidad de las soluciones tecnológicas.

Agenda



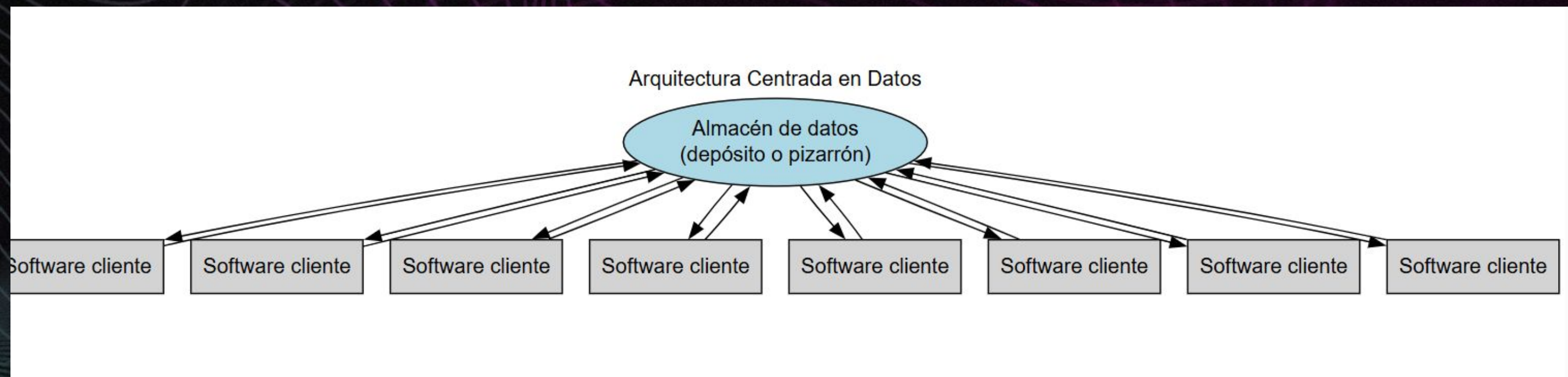
¡Dudas
proyecto?



Teoría

Arquitectura centrada en datos

Una arquitectura centrada en datos (Data-Centric Architecture) es un enfoque de diseño de sistemas donde el dato es el elemento central y principal de la arquitectura. En lugar de organizarse alrededor de aplicaciones, procesos o funciones, todo el ecosistema de TI se estructura para almacenar, compartir, gobernar y explotar los datos como un recurso estratégico





Características principales

01

Dato como activo primario

Los datos se consideran el núcleo de la organización, mientras que las aplicaciones son vistas como herramientas temporales para acceder y manipularlos.

02

Interoperabilidad:

Se busca un modelo de datos común y estandarizado que permita a múltiples aplicaciones y servicios consumir la información sin duplicaciones.

03

Reducción de silos:

Los sistemas dejan de almacenar datos de forma aislada (data silos) y se orientan hacia repositorios integrados, como data lakes, data warehouses o mallas de datos (data mesh).

04

Gobernanza y calidad de datos

Se enfatiza en la seguridad, consistencia, trazabilidad y control de acceso.

Ventajas

01

Mayor reutilización de datos entre diferentes procesos.

02

Reducción de redundancia y errores de inconsistencia.

03

Facilidad para la analítica y la inteligencia de negocios.

04

Escalabilidad: los sistemas pueden crecer alrededor de los datos sin rehacer aplicaciones enteras.

05

Enfoque estratégico: convierte al dato en el centro de la transformación digital.

Desventajas

01

Complejidad inicial en el diseño del modelo de datos común.

02

Costos asociados a la migración desde arquitecturas centradas en aplicaciones.

03

Resistencia cultural: muchos equipos están acostumbrados a priorizar aplicaciones en lugar de datos

04

Necesidad de gobernanza robusta para asegurar calidad y seguridad.

Repositorio y Pizarra

Repositorio Pasivo

Es un almacén centralizado donde los datos se guardan y los softwares clientes acceden a ellos cuando lo necesitan.

El repositorio no ejecuta lógica ni decide nada: solo almacena y responde solicitudes.

Los clientes tienen que "ir" al repositorio para leer o escribir datos.

Pizarra

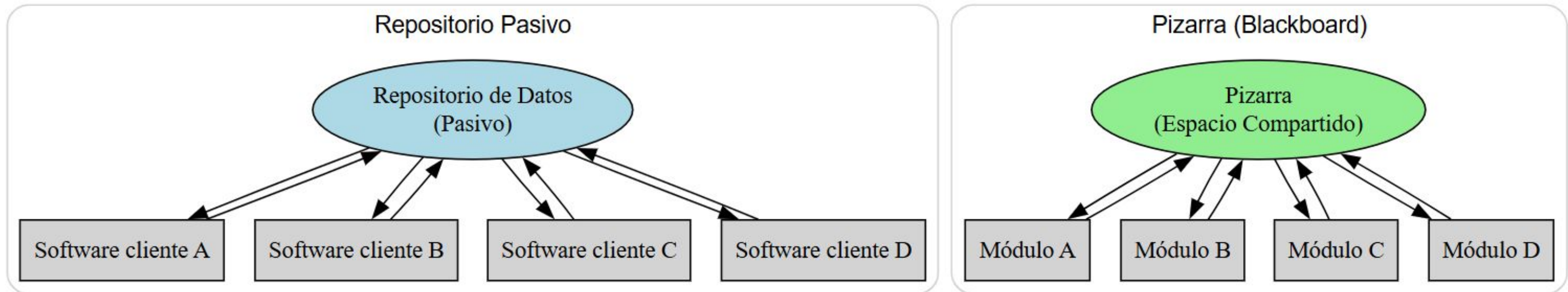
Es una arquitectura de coordinación donde varios módulos (softwares clientes) leen y escriben sobre un espacio compartido de datos (la "pizarra").

El dato compartido guía el comportamiento de los sistemas: cuando un cliente coloca información nueva, otros módulos pueden reaccionar automáticamente.

La pizarra actúa como un mecanismo de integración inteligente.

Repositorio y Pizarra

Comparación: Repositorio Pasivo vs Pizarra (Blackboard)



Repositorio Pasivo:

El repositorio es solo un depósito central.

Los softwares clientes leen y escriben, pero la lógica vive en los clientes.

Pizarra (Blackboard):

La pizarra es un espacio de trabajo activo y compartido.
Los módulos interactúan indirectamente: uno escribe, y otros reaccionan.

Esto fomenta coordinación e inteligencia colectiva.

Pizarra

se basa en un concepto muy humano: el de la resolución cooperativa de problemas. Imagina un equipo de expertos, cada uno con un conocimiento único, que se comunica a través de una pizarra compartida. No se hablan directamente, sino que cada uno lee el estado actual de la pizarra y, cuando ve la oportunidad de contribuir, escribe su solución parcial.

- La Pizarra (Blackboard): Es una memoria o un almacén de datos global donde se guarda el estado del problema. Los datos aquí pueden ser estructuras de conocimiento, hipótesis o soluciones parciales.
- Fuentes de Conocimiento (Knowledge Sources): Son módulos independientes que contienen la lógica o "expertos" para resolver el problema. Cada módulo se especializa en una tarea específica y actúa sobre los datos de la pizarra. No se comunican entre sí directamente, solo a través de la pizarra.
- El Controlador (Control Component): Es el que decide qué fuente de conocimiento debe actuar y en qué momento. Supervisa el estado de la pizarra y activa los módulos de conocimiento más apropiados para avanzar hacia la solución.

Repositorio pasivo

El patrón repositorio pasivo es un modelo de arquitectura más sencillo y común. Aquí, el enfoque está en un repositorio central de datos, pero a diferencia de la pizarra, la lógica no está en los datos, sino en los componentes que interactúan con ellos.

- El Repositorio (The Repository): Es un almacén de datos central, como una base de datos o un sistema de archivos. Es pasivo porque no tiene control sobre quién accede a los datos o cuándo.
- Los Clientes (Clients): Son los módulos o componentes que contienen toda la lógica del negocio. Ellos son los que inician las operaciones, solicitan datos del repositorio, los procesan y guardan los resultados. El repositorio simplemente responde a las peticiones.

CLIENTE - SERVIDOR

CLIENTE - SERVIDOR

Arquitectura Cliente-servidor, también conocida como Modelo Cliente-servidor o simplemente Cliente-servidor. Esta arquitectura consiste básicamente en un cliente que realiza peticiones a otro programa (el servidor) que le da respuesta. Aunque esta idea se puede aplicar a programas que se ejecutan sobre una sola computadora es más ventajosa en un sistema operativo multiusuario distribuido a través de una red de computadoras. La interacción cliente-servidor es el soporte de la mayor parte de la comunicación por redes. Ayuda a comprender las bases sobre las que están contruidos los algoritmos distribuidos.

Partes que componen el sistema

01

Cliente

Programa ejecutable que participa activamente en el establecimiento de las conexiones. Envía una petición al servidor y se queda esperando por una respuesta. Su tiempo de vida es finito una vez que son servidas sus solicitudes, termina el trabajo.

02

Servidor

Es un programa que ofrece un servicio que se puede obtener en una red. Acepta la petición desde la red, realiza el servicio y devuelve el resultado al solicitante. Al ser posible implantarlo como aplicaciones de programas, puede ejecutarse en cualquier sistema donde exista TCP/IP y junto con otros programas de aplicación. El servidor comienza su ejecución antes de comenzar la interacción con el cliente. Su tiempo de vida o de interacción es “interminable”



Características de la arquitectura Cliente-Servidor

1. Distribución de tareas (Separación de roles)

Cliente: Es el que solicita los servicios. Su rol principal es la presentación (interfaz de usuario) y la gestión de la interacción con el usuario. Es decir, se encarga de cómo se ven los datos y cómo el usuario puede interactuar con ellos.

Servidor: Es el que proporciona los servicios. Su rol es centralizar la lógica de negocio, los datos y los recursos. Responde a las peticiones del cliente, procesa la información y la envía de vuelta.

2. Centralización de recursos

Los recursos (como bases de datos, aplicaciones, archivos compartidos y otros servicios) están centralizados en uno o varios servidores. Esto facilita la administración, la seguridad y el respaldo de los datos.

Al tener los datos en un solo lugar, es más sencillo aplicar políticas de seguridad y realizar copias de seguridad de forma regular.



Características de la arquitectura Cliente-Servidor

3. Escalabilidad

La arquitectura cliente-servidor permite un crecimiento flexible. Si se necesita más capacidad para manejar más clientes o peticiones, se pueden añadir más servidores.

La escalabilidad puede ser horizontal (añadiendo más máquinas a la capa de servidores) o vertical (mejorando la capacidad de una máquina existente).

4. Mantenimiento y actualización

El mantenimiento y las actualizaciones se pueden realizar en el servidor de forma independiente. Por ejemplo, si se necesita actualizar el software de la base de datos o la lógica de negocio, solo se tiene que hacer en el servidor.

Esto reduce significativamente la carga de trabajo y el tiempo de inactividad, ya que no es necesario actualizar cada máquina cliente individualmente.



Características de la arquitectura Cliente-Servidor

5. Independencia de la plataforma

El cliente y el servidor pueden funcionar en plataformas de hardware y sistemas operativos diferentes.

Por ejemplo, un cliente con un sistema operativo Windows puede solicitar servicios a un servidor que funciona con Linux. Esta interoperabilidad se logra mediante protocolos de red estandarizados como TCP/IP.

6. Seguridad

La seguridad es más fácil de gestionar y controlar, ya que se implementa principalmente en el servidor. El servidor puede manejar la autenticación, la autorización y el control de acceso a los recursos.

Esto evita que los datos sensibles se distribuyan por la red y reduce el riesgo de accesos no autorizados.

7. Flexibilidad y reutilización

La separación de la interfaz de usuario de la lógica de negocio permite que un mismo servidor proporcione servicios a diferentes tipos de clientes (por ejemplo, una aplicación web, una aplicación de escritorio o una aplicación móvil). Esto fomenta la reutilización del código del servidor.

CAPAS EN UNA ARQUITECTURA CLIENTE-SERVIDOR

Aunque la arquitectura cliente-servidor se describe fundamentalmente como un modelo de dos partes, la implementación de sistemas complejos suele organizarse en capas. Esto ayuda a separar las diferentes responsabilidades, haciendo que el sistema sea más fácil de mantener, escalar y actualizar. Las tres capas principales son:

CAPA DE PRESENTACIÓN (CLIENTE):

(cliente): Es la interfaz con la que el usuario interactúa. Su función es mostrar la información y recibir las acciones del usuario. Piensa en tu navegador web, una aplicación de escritorio o una app móvil.

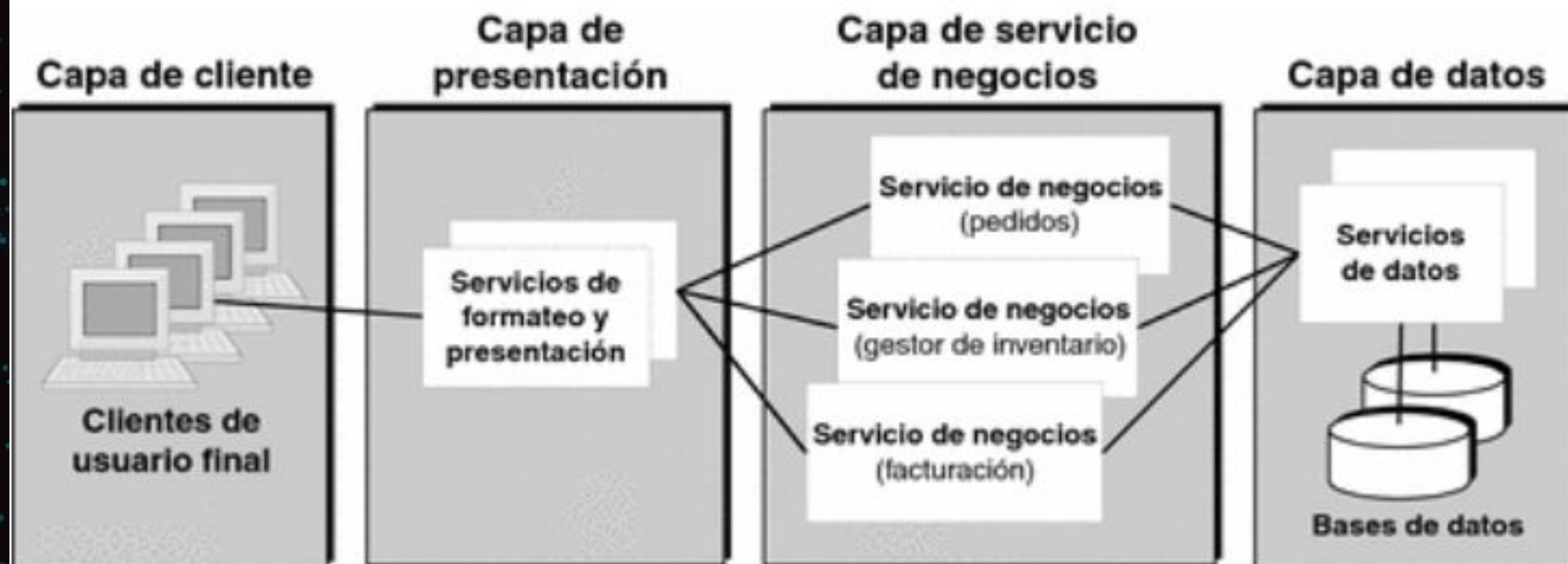


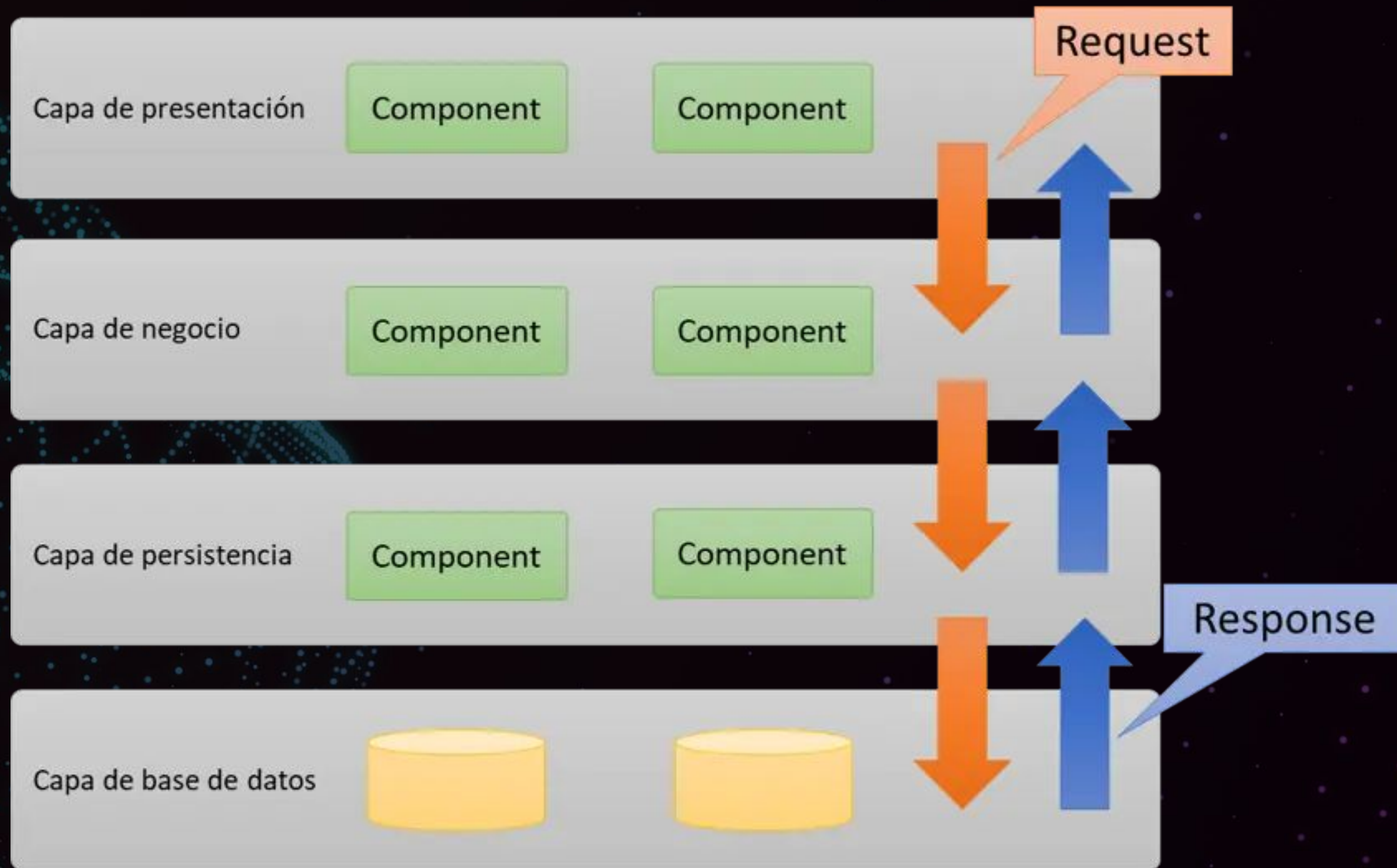
CAPA DE LÓGICA DE NEGOCIO (SERVIDOR):

Contiene las reglas y la lógica del negocio. Es el "cerebro" del sistema, donde se procesan las solicitudes, se realizan cálculos y se aplican las reglas definidas. Por ejemplo, en un sistema de banca en línea, esta capa procesa las transferencias, verifica los saldos y las valida.

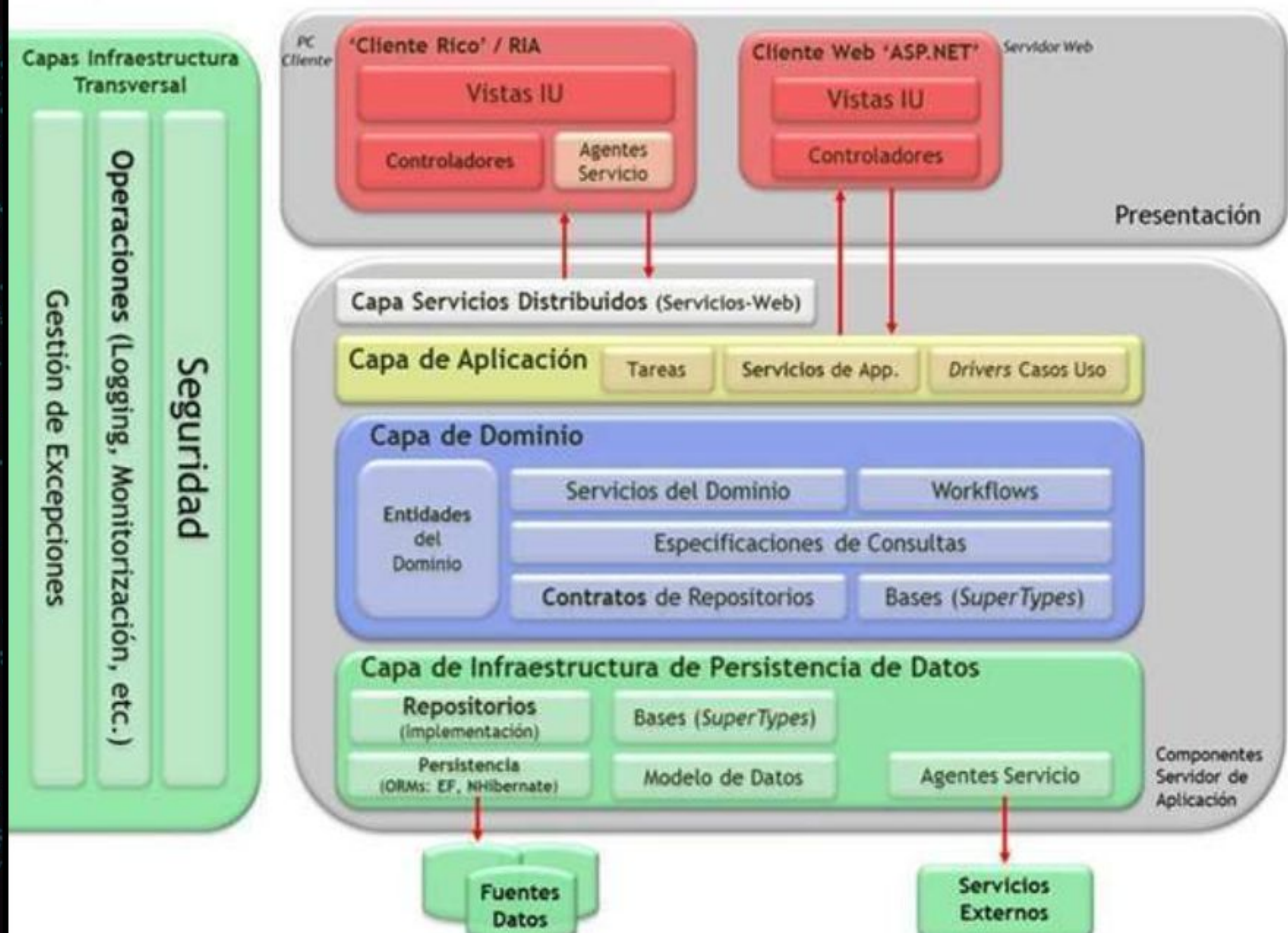
CAPA DE DATOS (SERVIDOR):

Almacena, gestiona y recupera los datos. Es la base de datos del sistema, que puede ser un servidor de bases de datos relacionales como MySQL o una base de datos no relacional como MongoDB. Esta capa asegura que los datos sean coherentes y estén disponibles cuando se necesiten.





Arquitectura N-Capas con Orientación al Dominio



Tuberías y filtros

TUBERÍAS Y FILTROS

La arquitectura de Tuberías y Filtros es especialmente adecuada para sistemas de procesamiento de datos, donde los datos se reciben, transforman y envían a otros sistemas o procesos. Esta arquitectura permite la separación de tareas, lo que hace que el sistema sea más modular y fácil de mantener. También permite la reutilización de filtros en diferentes canales de procesamiento.

Componentes de la arquitectura de tuberías y filtros

Tubería

Las tuberías son canales por los que fluyen los datos entre filtros. En un sistema de procesamiento de datos, los datos suelen pasar entre filtros en forma de mensajes. Las tuberías pueden implementarse mediante técnicas como memoria compartida, sockets o colas de mensajes.

Filtros

Los filtros son pasos de procesamiento que reciben datos de una o más tuberías, realizan un procesamiento y envían los datos procesados a una o más tuberías.

Los filtros son independientes entre sí y pueden combinarse de diferentes maneras para crear diversas tuberías de procesamiento.

Componentes de la arquitectura de tuberías y filtros

Tubería

Las tuberías son canales por los que fluyen los datos entre filtros. En un sistema de procesamiento de datos, los datos suelen pasar entre filtros en forma de mensajes. Las tuberías pueden implementarse mediante técnicas como memoria compartida, sockets o colas de mensajes.

Filtros

Los filtros son pasos de procesamiento que reciben datos de una o más tuberías, realizan un procesamiento y envían los datos procesados a una o más tuberías.

Los filtros son independientes entre sí y pueden combinarse de diferentes maneras para crear diversas tuberías de procesamiento.

Los filtros se pueden clasificar en tres categorías:

Filtros de origen : reciben datos de entrada de un sistema o fuente externa y los pasan al siguiente filtro en la tubería.

Filtros de procesamiento : reciben datos de entrada de una o más tuberías, realizan algún procesamiento en los datos y envían los datos procesados a una o más tuberías.

Filtros de sumidero : reciben datos del filtro anterior en la tubería, los procesan y los envían a un sistema externo o sumidero.

Ventajas de la arquitectura de tuberías y filtros

1. Separación de preocupaciones: La arquitectura permite la separación de preocupaciones entre los diferentes pasos de procesamiento. Cada filtro es responsable de una tarea específica, lo que hace que el sistema sea más modular y fácil de mantener.

2. Reutilización: Los filtros se pueden reutilizar en diferentes canales de procesamiento, lo que reduce el tiempo de desarrollo y mejora el rendimiento general del sistema.

Escalabilidad:

3. La arquitectura facilita la escalabilidad del sistema. Se pueden añadir filtros adicionales al pipeline para gestionar mayores requisitos de procesamiento.

4. Tolerancia a fallos: La arquitectura proporciona tolerancia a fallos aislando los filtros entre sí. Si un filtro falla, no afecta a los demás filtros de la tubería.

Pruebas

5. La arquitectura facilita la prueba de filtros individuales, ya que pueden probarse de forma aislada.

Rendimiento

Ejempl

O

Consideremos un ejemplo de un sistema de procesamiento de datos simple que procesa pedidos de clientes. El sistema consta de cuatro filtros: el receptor de pedidos, el analizador de pedidos, el validador de pedidos y el emisor de pedidos. El receptor de pedidos recibe los pedidos de los clientes y los envía al analizador de pedidos. El analizador de pedidos analiza los datos del pedido y los envía al validador de pedidos. El validador de pedidos valida los datos del pedido y los envía al emisor de pedidos. Finalmente, el emisor de pedidos escribe los datos validados en una base de datos.

El flujo de datos entre los filtros se muestra en el siguiente diagrama

Ejempl

O

Consideremos un ejemplo de un sistema de procesamiento de datos simple que procesa pedidos de clientes. El sistema consta de cuatro filtros: el receptor de pedidos, el analizador de pedidos, el validador de pedidos y el emisor de pedidos. El receptor de pedidos recibe los pedidos de los clientes y los envía al analizador de pedidos. El analizador de pedidos analiza los datos del pedido y los envía al validador de pedidos. El validador de pedidos valida los datos del pedido y los envía al emisor de pedidos. Finalmente, el emisor de pedidos escribe los datos validados en una base de datos.

El flujo de datos entre los filtros se muestra en el siguiente diagrama

Receptor de órdenes -> Analizador de órdenes -> Validador de órdenes -> Escritor de órdenes

Ejempl

O

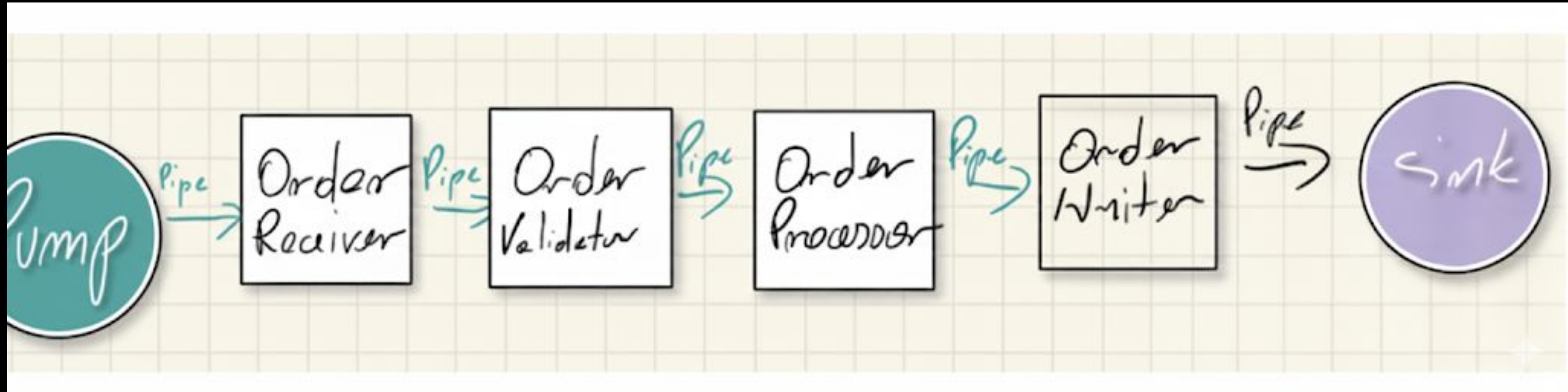
Consideremos un ejemplo de un sistema de procesamiento de datos simple que procesa pedidos de clientes. El sistema consta de cuatro filtros: el receptor de pedidos, el analizador de pedidos, el validador de pedidos y el emisor de pedidos. El receptor de pedidos recibe los pedidos de los clientes y los envía al analizador de pedidos. El analizador de pedidos analiza los datos del pedido y los envía al validador de pedidos. El validador de pedidos valida los datos del pedido y los envía al emisor de pedidos. Finalmente, el emisor de pedidos escribe los datos validados en una base de datos.

El flujo de datos entre los filtros se muestra en el siguiente diagrama

Receptor de órdenes -> Analizador de órdenes -> Validador de órdenes -> Escritor de órdenes

Ejempl

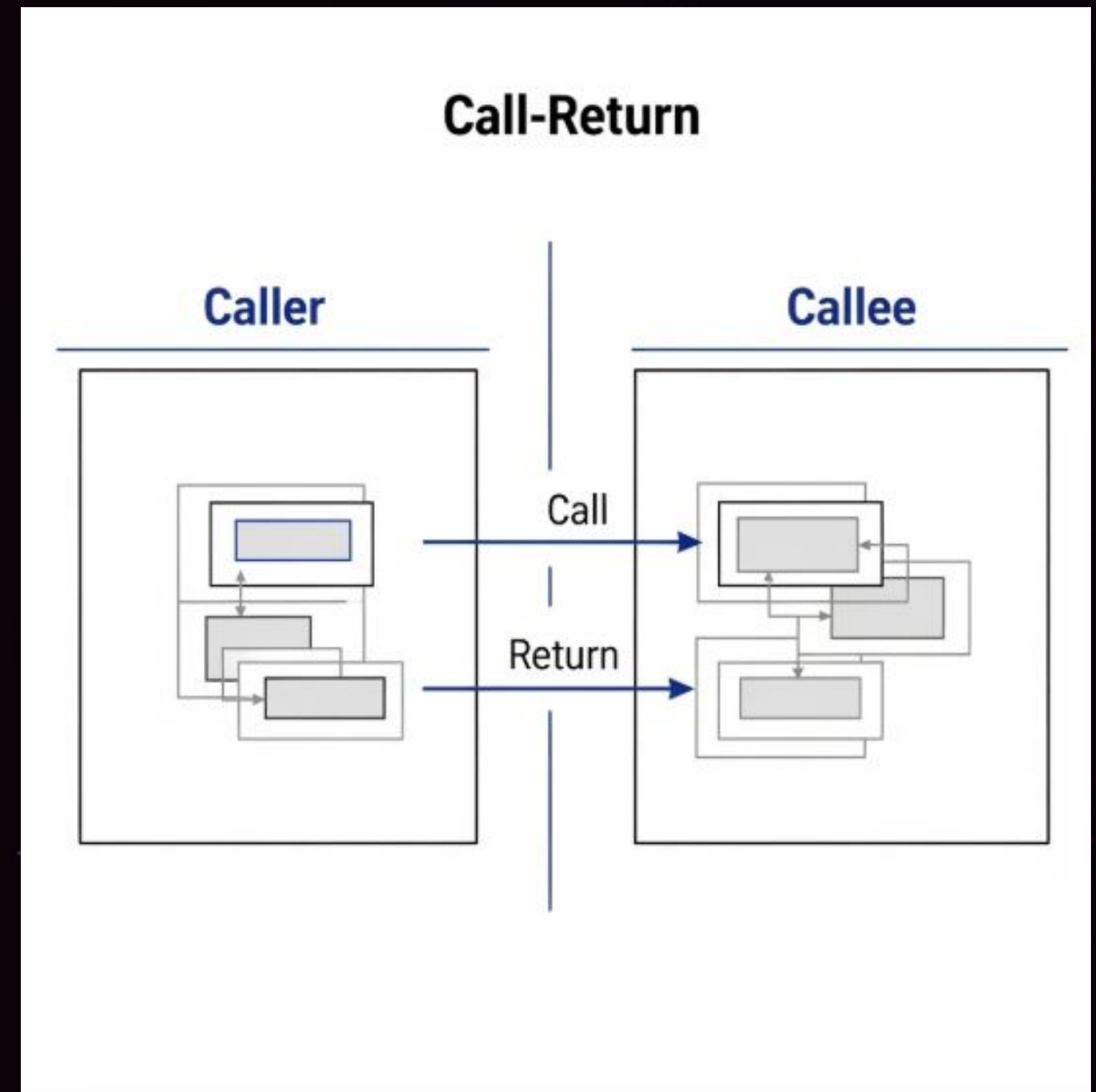
o



LLAMADA - RETORNO

LLAMADA - RETORNO

Es un estilo de diseño de sistemas de software que organiza la estructura del programa en módulos jerárquicos. Estos módulos se comunican entre sí a través de un mecanismo de "llamada" y "retorno", donde un módulo de nivel superior invoca a uno de nivel inferior para realizar una tarea específica.



CARACTERÍSTICAS Y COMPONENTES

Este estilo se basa en la división de un programa en unidades lógicas, como procedimientos, funciones o métodos. Los principales componentes son:

- Módulos de Llamada (Calling Modules): Son los módulos de nivel superior que tienen la responsabilidad de invocar a otros módulos.
- Módulos de Retorno (Returning Modules): Son los módulos de nivel inferior que son llamados para ejecutar una tarea. Después de completarla, devuelven el control y, a menudo, un resultado al módulo que los invocó.
- Pila de Llamadas (Call Stack): Es la estructura de datos fundamental que soporta esta arquitectura. La pila almacena la dirección de retorno de cada llamada, permitiendo que el programa regrese al punto exacto donde fue invocado.

Ventajas:

- **Modularidad y Reutilización:** Permite dividir un sistema complejo en partes más pequeñas y manejables. Esto facilita la reutilización de código, ya que una función puede ser llamada desde múltiples lugares.
- **Facilidad de Mantenimiento:** Al aislar las funcionalidades en módulos, es más sencillo depurar, modificar y mantener el código sin afectar otras partes del sistema.
- **Estructura Jerárquica:** Fomenta un diseño claro y organizado, donde las dependencias fluyen de arriba hacia abajo, lo que facilita la comprensión del flujo del programa.

Desventajas:

Acoplamiento: Puede generar un alto acoplamiento entre los módulos si las dependencias son demasiado rígidas. Un cambio en un módulo de nivel inferior podría requerir cambios en los módulos que lo llaman.

Flexibilidad Limitada: No es ideal para sistemas donde los módulos necesitan comunicarse de manera no jerárquica o asíncrona, como en arquitecturas basadas en eventos o microservicios.

Carga de la Pila de Llamadas: Un número excesivo de llamadas anidadas (recursión profunda) puede agotar la memoria de la pila, causando un error de desbordamiento de pila (stack overflow).

BASADA EN
EVENTOS

BASADA EN EVENTOS

La arquitectura basada en eventos (EDA, por sus siglas en inglés) es un patrón de diseño de software donde los componentes reaccionan a cambios de estado llamados eventos. Este patrón se basa en un modelo de publicador-consumidor desacoplado, donde los productores de eventos emiten eventos a un enrutador, que luego los dirige a los consumidores apropiados. Esto permite crear sistemas más escalables, flexibles y con capacidad de respuesta que funcionan en tiempo real, ya que los componentes no necesitan conocerse directamente para comunicarse y pueden operar de forma independiente.

¿COMO FUNCIONA?

1. Evento:

Ocurre un suceso en el sistema, como un artículo agregado al carrito de compra.

2. Productor:

La aplicación o servicio que detecta este evento publica un mensaje que lo describe.

3. Enrutador:

Un componente central, como un bus de eventos, recibe este evento y lo filtra.

4. Consumidor:

Los servicios que están suscritos a ese tipo de evento lo reciben y reaccionan de

COMPONENTES CLAVE

- Productor: El componente que genera el evento.
- Consumidor: El componente que recibe y procesa el evento.
- Enrutador/Broker de eventos: El intermediario que recibe los eventos del productor y los enruta a los consumidores adecuados

CARACTERÍSTICAS PRINCIPALES

- Desacoplamiento:

Los productores y consumidores no necesitan conocerse directamente, lo que facilita la escalabilidad y la actualización independiente de los componentes.

- Asincronía:

La comunicación se realiza de forma asíncrona; los productores no esperan a que los consumidores reciban el evento.

- Escalabilidad:

Es ideal para manejar sistemas que generan o consumen una gran cantidad de datos y para integrar diversos microservicios.

- Flexibilidad:

Beneficios de una arquitectura basada en eventos

Escalado y errores por separado

Al desacoplar los servicios, estos solo conocen el enrutador de eventos, no los demás. Esto significa que sus servicios son interoperables, pero si un servicio tiene un error, el resto seguirá funcionando. El enrutador de eventos actúa como un búfer elástico que se adapta a los aumentos repentinos de las cargas de trabajo.

Auditar con facilidad

Un enrutador de eventos actúa como una ubicación centralizada para auditar su aplicación y definir políticas. Estas políticas pueden restringir quién puede publicar y suscribirse a un enrutador, y controlar qué usuarios y recursos tienen permiso para acceder a sus datos. También puede cifrar sus eventos tanto en tránsito como en reposo.

Desarrollar con agilidad

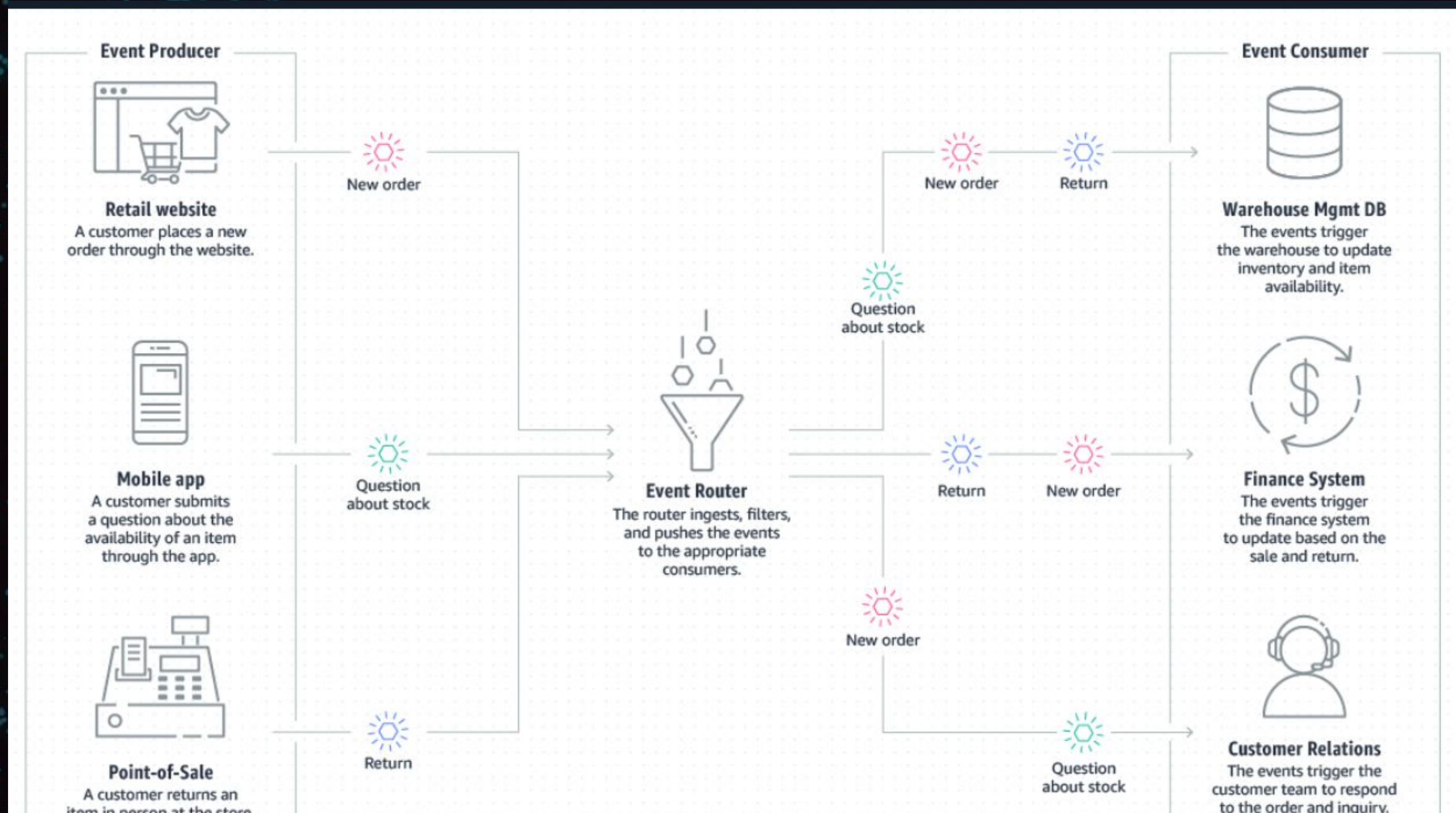
Ya no es necesario escribir código personalizado para sondear, filtrar y enrutar eventos; el enrutador de eventos filtrará y enviará automáticamente los eventos a los consumidores. El enrutador también elimina la necesidad de una fuerte coordinación entre los servicios productores y consumidores, acelerando su proceso de desarrollo.

Reducción de costos

Las arquitecturas basadas en eventos son de tipo push, por lo que todo ocurre bajo demanda cuando el evento se presenta en el enrutador. De este modo, no tiene que pagar por el sondeo continuo para comprobar si hay un evento. Esto significa menos consumo de ancho de banda de la red, menos consumo de CPU, menos capacidad de flota ociosa y menos handshakes SSL/TLS.

CÓMO FUNCIONA: ARQUITECTURA DE EJEMPLO

ESTE ES UN EJEMPLO DE ARQUITECTURA BASADA EN EVENTOS PARA UN SITIO DE COMERCIO ELECTRÓNICO. ESTA ARQUITECTURA PERMITE QUE EL SITIO REACCIONE A LOS CAMBIOS PROCEDENTES DE DIVERSAS FUENTES DURANTE LOS MOMENTOS DE MAYOR DEMANDA, SIN QUE SE BLOQUEE LA APLICACIÓN NI SE SOBREAPROVISIONEN RECURSOS.



Cuándo utilizar esta arquitectura

Replicación de datos entre cuentas y regiones

Puede utilizar una arquitectura basada en eventos para coordinar sistemas entre equipos que operan y se implementan en diferentes regiones y cuentas. Al utilizar un enrutador de eventos para transferir datos entre sistemas, puede desarrollar, escalar e implementar servicios independientemente de otros equipos.

Procesamiento en paralelo y distribución ramificada

Si tiene muchos sistemas que necesitan operar en respuesta a un evento, puede utilizar una arquitectura basada en eventos que permita distribuir el evento sin tener que escribir código personalizado para enviarlo a cada consumidor. El enrutador enviará el evento a los sistemas, cada uno de los cuales puede procesar el evento en paralelo con un propósito diferente.

Supervisión del estado de los recursos y alertas

En lugar de comprobar continuamente los recursos, puede utilizar una arquitectura basada en eventos para supervisar y recibir alertas sobre cualquier anomalía, cambio o actualización. Estos recursos pueden incluir buckets de almacenamiento, tablas de bases de datos, funciones sin servidor, nodos de computación, etc.

Integración de sistemas heterogéneos

Si tiene sistemas que se ejecutan en diferentes pilas, puede utilizar una arquitectura basada en eventos para compartir información entre ellos sin acoplarse. El enrutador de eventos establece la indirección y la interoperabilidad entre los sistemas, para que puedan intercambiar mensajes y datos sin dejar de ser independientes.

CASOS PRÁCTICOS DE LA ARQUITECTURA BASADA EN EVENTOS

Orquestación de microservicios



Procesamiento y analíticas de datos en tiempo real



Plataformas de comercio electrónico



Automatización de flujos de trabajo y gestión de procesos empresariales



Sincronización y replicación de datos



Informática sin servidor



Servicios financieros



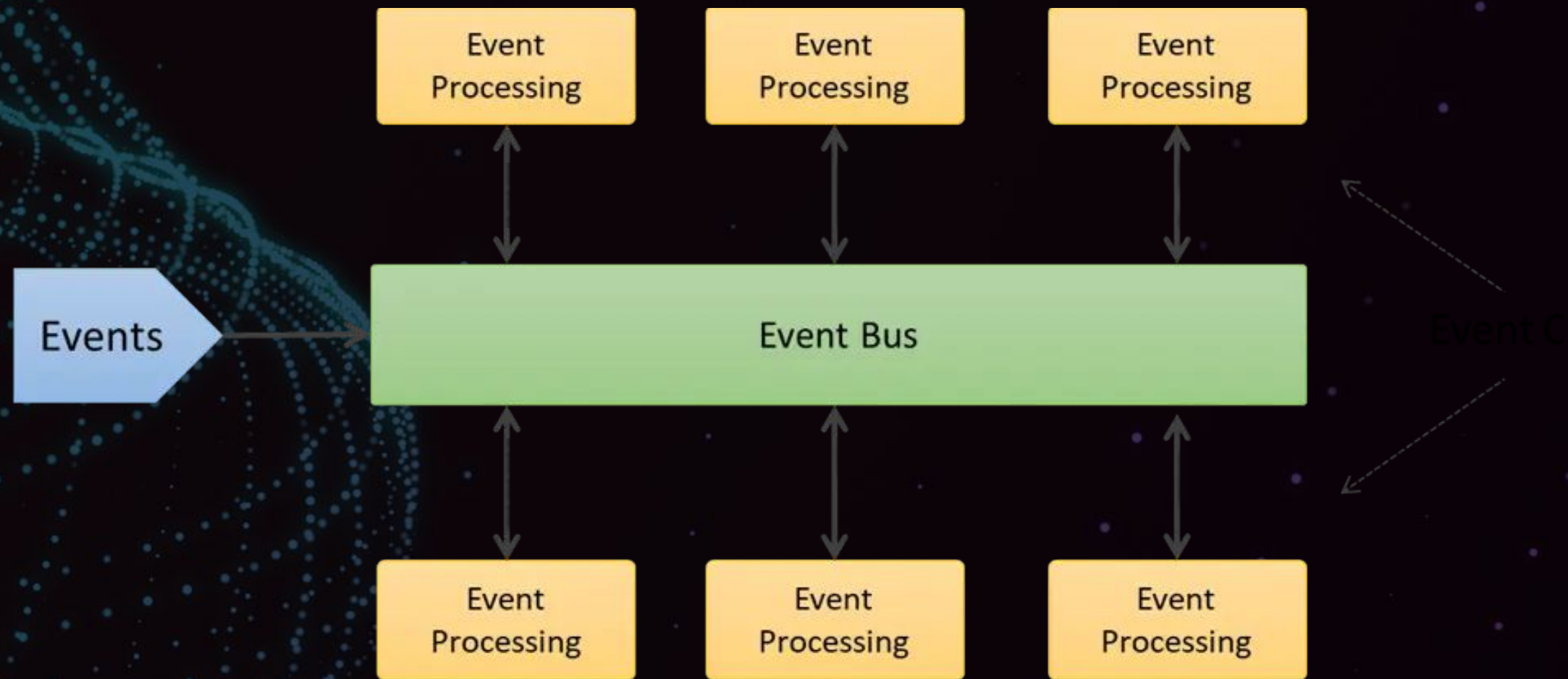
EJEMPLO

UN ESTUDIANTE UNIVERSITARIO PIDE UNA PIZZA A TRAVÉS DE UNA APP DE ENTREGA DE COMIDA, COMO UBER EATS. LA APP CAPTURA SU INFORMACIÓN BÁSICA (NOMBRE, DIRECCIÓN, INFORMACIÓN DE PAGO Y PEDIDO) Y PUBLICA EL EVENTO COMO "PEDIDO DE PIZZA".

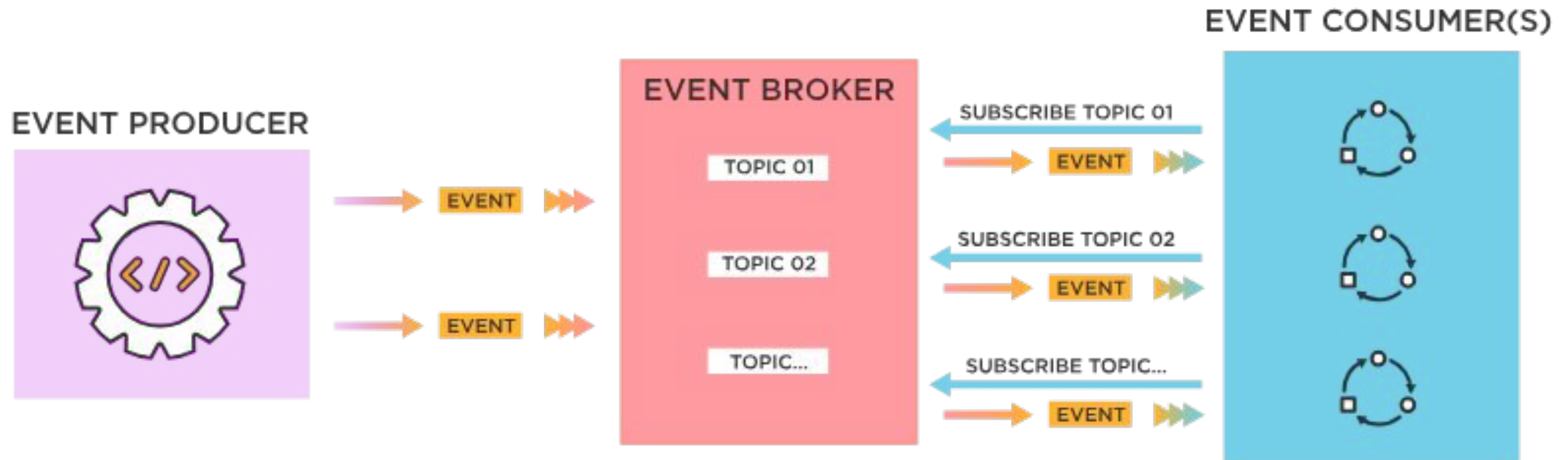
LA PIZZERÍA SE SUSCRIBE AL EVENTO, CUMPLE CON EL PEDIDO, Y PUBLICA SU PROPIO EVENTO COMO "PEDIDO LISTO" DE NUEVO EN EL SERVICIO DE ENTREGA DE COMIDA.

LUEGO, EL SERVICIO ASIGNA UN CONDUCTOR PARA LA ENTREGA, PROGRAMA UNA HORA DE LLEGADA, Y LE AVISA AL CLIENTE QUE SU PIZZA ESTÁ EN CAMINO.

EJEMPLO



EJEMPLO



EJEMPLO



CONCEPTOS CLAVE APRENDIDOS

- Patron de diseño estructurales:
 - Adapter
 - Proxy
 - Facade
 - Decorator



Ejemplo práctico

La cafetería del instituto está colapsada en los recreos. Los alumnos pierden todo el tiempo haciendo fila y los encargados se confunden con los pedidos. Se te pide diseñar la **Arquitectura Cliente-Servidor** para una aplicación que solucione esto.

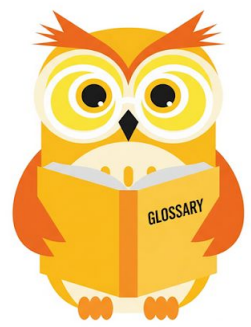
1. El Cliente (Front-end / Interfaz)

- **Dispositivo:** Tablet en el mostrador o App en el móvil del alumno.
- **Responsabilidades:** * Mostrar el menú (fotos, precios, disponibilidad).
 - Recoger el pedido del usuario (ej: "1 bocadillo de jamón y 1 zumo").
 - Validar que el usuario haya iniciado sesión.
 - Enviar la solicitud de pedido al servidor.

2. El Servidor (Back-end / Lógica y Datos)

- **Ubicación:** Un ordenador central (o la nube).
- **Responsabilidades:**
 - **Procesar el pedido:** Verificar si hay stock (¿quedan bocadillos?).
 - **Seguridad:** Comprobar que el alumno tiene saldo en su cuenta.
 - **Almacenamiento:** Guardar el historial de ventas en la Base de Datos.
 - **Respuesta:** Enviar una confirmación al cliente ("Pedido aceptado, turno #45").





Practiquemos lo aprendido

Diseñar un diagrama de flujo de datos donde un texto en bruto (input) pase por una serie de **Filtros** conectados por **Tuberías** hasta llegar a un estado de 'Aprobado' o 'Rechazado' (output)."

Vuestro sistema debe incluir obligatoriamente estos 4 Filtros (en el orden que consideréis lógico):

1. **Filtro de Idioma:** Identifica si el texto está en el idioma correcto.
2. **Filtro de 'Blacklist':** Busca palabras prohibidas o insultos.
3. **Filtro de Verificación:** Compara el texto con una base de datos de noticias confirmadas.
4. **Filtro de Formato:** Elimina etiquetas HTML, emoticonos excesivos o errores de código.



Nombre de la actividad:

Cantidad de participantes:

Doy fe que esta actividad está
planificada en dtt (Sí/No):

Hora de inicio:

Hora de fin:

Duración (min):

Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)



DUDAS ¿?



**¡GRACIAS POR SU
ATENCIÓN!**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA

¿Preguntas? ¿Dudas?